Complete validation of the exceptional two-orbit example

Human Compression project

Additive author edition; pending independent review

The entire program below reconstructs the literal affine maps, the 144-flag input, and its 2592-flag regular core. It checks every flag and every pair of color subsets, the full automorphism orbit argument, both embedded variance tests and their explicit witness. It also verifies the complete pair formulas used in the written proof. It is bounded validation of that proof, not a classification or a general geometric premise. The operational replay driver and its corruption controls are inventoried separately. There are no new Lean declarations in this addition.

```
#!/usr/bin/env python3
"""Literal F3 affine graphs, all flag intersections, and embedded variance groups."""
import argparse, hashlib, json
from itertools import product
from pathlib import Path

def need(v,m):
    if not v: raise RuntimeError(m)

def compose(x,y):
    t,f=x;u,g=y
    return (tuple((t[i]+(1 if f[i]>0 else -1)*u[abs(f[i])-1])%3 for i in range(2)),
            tuple((1 if f[i]>0 else -1)*g[abs(f[i])-1] for i in range(2)))

def closure(gens,identity,mul):
    out=[identity];seen={identity}
    for x in out:
        for g in gens:
            y=mul(g,x)
            if y not in seen:seen.add(y);out.append(y)
    return out

def components(adj,mask):
    todo=set(range(len(adj)));bits=[];ids=[-1]*len(adj)
    while todo:
        x=min(todo);todo.remove(x);q=[x];members={x}
        for x in q:
            for i in range(4):
                if mask>>i&1:
                    y=adj[x][i]
                    if y not in members:members.add(y);todo.remove(y);q.append(y)
        for x in q:ids[x]=len(bits)
        bits.append(sum(1<<x for x in q))
    return bits,ids

def orbit(c,q):return c[0][c[1][q]]

def graph_check(adj):
    n=len(adj)
    for x,row in enumerate(adj):
        need(len(row)==4 and len(set(row))==4 and x not in row,'Four distinct nonloop neighbors')
        for i in range(4):
            need(adj[row[i]][i]==x,'Involutions')
            for j in range(i+2,4):need(adj[row[i]][j]==adj[row[j]][i],'Distant commutation')
    C={m:components(adj,m) for m in range(16)}
    need(len(C[15][0])==1,'Connectedness')
    checks=0
    for q in range(n):
        for i in range(16):
            for j in range(16):
                need(orbit(C[i],q)&orbit(C[j],q)==orbit(C[i&j],q),'Path intersection at '+str((q,i,j)))
```

```
                checks+=1
    return C,checks

def main():
    p=argparse.ArgumentParser();p.add_argument('--out',type=Path,required=True);args=p.parse_args()
    e=((0,0),(1,2));a=((0,0),(-1,2));b=((0,1),(1,-2));d=((0,0),(2,1))
    ap=compose(compose(d,a),d);bp=compose(compose(d,b),d)
    H=closure([a,b,d],e,compose);hi={h:i for i,h in enumerate(H)}
    need(len(H)==72,'Affine group order')
    # Evaluate each affine map on its nine literal points; faithfulness is checked.
    points=list(product(range(3),repeat=2));pi={x:i for i,x in enumerate(points)}
    def action(h,z):
        t,f=h
        return tuple((t[i]+(1 if f[i]>0 else -1)*z[abs(f[i])-1])%3 for i in range(2))
    perms={h:tuple(pi[action(h,x)] for x in points) for h in H}
    need(len(set(perms.values()))==72,'Literal nine-point faithfulness')
    for h,k in product(H,repeat=2):
        need(compose(h,k) in hi,'Closure')
        need(perms[compose(h,k)]==tuple(perms[h][perms[k][j]] for j in range(9)),'Composition bridge')
    P=[[2*hi[h]+(eps^1),2*hi[compose(a if eps==0 else ap,h)]+eps,
        2*hi[compose(b,h)]+eps,2*hi[compose(d,h)]+(eps^1)] for h in H for eps in range(2)]
    PC,pip=graph_check(P)
    autos=[]
    for k in H:
        f=[2*hi[compose(h,k)]+eps for h in H for eps in range(2)]
        need(len(set(f))==len(P),'Right action permutation')
        need(all(f[P[q][i]]==P[f[q]][i] for q in range(len(P)) for i in range(4)),'Right action automorphism')
        autos.append(f)
    need({f[0] for f in autos}==set(range(0,144,2)),'First true flag orbit')
    need({f[1] for f in autos}==set(range(1,144,2)),'Second true flag orbit')
    need(all(orbit(PC[6],q).bit_count()==(4 if q%2==0 else 6) for q in range(144)),
         'No automorphism swaps the two medial sizes')
    for mask in [7,14]:
        need({PC[mask][1][f[0]] for f in autos}==set(range(len(PC[mask][0]))),'Uniform full end orbit')
    need(len({PC[6][1][f[0]] for f in autos})==18,'Digon section orbit')
    need(len({PC[6][1][f[1]] for f in autos})==12,'Triangle section orbit')
    # Connected rooted mix: no ambient component is silently added.
    pairs=[(0,1)];qi={pairs[0]:0};Q=[]
    for x,y in pairs:
        row=[]
        for i in range(4):
            z=(P[x][i],P[y][i])
            if z not in qi:qi[z]=len(pairs);pairs.append(z)
            row.append(qi[z])
        Q.append(row)
    need(len(Q)==2592,'Regular mix order')
    QC,qip=graph_check(Q)
    pairmul=lambda x,y:(compose(x[0],y[0]),compose(x[1],y[1]))
    U=(a,ap);W=(ap,a);V=(b,b);Vp=(bp,bp);DD=(d,d)
    K=closure([U,V,DD],(e,e),pairmul)
    need(len(K)==1296,'Even subgroup order')
    need({(H[x//2],H[y//2]) for x,y in pairs if x%2==0}==set(K),'Even pair model')
    reggens=[]
    for g,k in [U,V,DD]:
        f=[qi[(2*hi[compose(H[x//2],g)]+x%2,2*hi[compose(H[y//2],k)]+y%2)] for x,y in pairs]
        reggens.append(f)
    reggens.append([qi[(y,x)] for x,y in pairs])
    for f in reggens:
        need(len(set(f))==len(Q),'Regular action permutation')
        need(all(f[Q[q][i]]==Q[f[q]][i] for q in range(len(Q)) for i in range(4)),'Regular action commutes')
    todo=[0];seen={0}
    for q in todo:
        for f in reggens:
            if f[q] not in seen:seen.add(f[q]);todo.append(f[q])
    need(len(seen)==2592,'Transitive automorphism action on mix')
    # Its size equals the actual connection group, confirming the regular core.
    marks=[tuple(row[i] for row in P) for i in range(4)]
    pmul=lambda g,h:tuple(g[h[i]] for i in range(len(P)))
    monodromy=closure(marks,tuple(range(len(P))),pmul)
    need(len(monodromy)==2592,'Actual regular core degree')
    KA=set(closure([U,W,V],(e,e),pairmul));KB=set(closure([U,V,Vp],(e,e),pairmul))
    fa=set();fb=set()
    for u,w,v,al,be in product(range(2),range(2),range(2),range(3),range(3)):
        sign=lambda t:1 if t%2==0 else -1
        fa.add((((0,al),(sign(u),2*sign(w+v))),((0,be),(sign(w),2*sign(u+v)))))
```

```
        fb.add((((al,v),(sign(u+w),2*sign(v))),((w,be),(sign(w),2*sign(u+v)))))
    need(KA==fa and len(KA)==72,'Full facet pair formula')
    need(KB==fb and len(KB)==72,'Full vertex pair formula')
    def fiber(q,mask):
        x,y=pairs[q];need(H[y//2]==e,'Specified second roots use identity')
        bits=orbit(QC[mask],q)
        return {H[pairs[z][1]//2] for z in range(len(Q)) if bits>>z&1 and pairs[z][0]==x}
    q0=0;q1=Q[0][0]
    A0,B0,C0=[fiber(q0,j) for j in [7,14,6]]
    A1,B1,C1=[fiber(q1,j) for j in [7,14,6]]
    AF={((0,c),(s,2*s)) for s in [1,-1] for c in range(3)}
    BF0={((u,c),((-1)**u,2*(-1)**u)) for u in range(2) for c in range(3)}
    BF1={((c,u),((-1)**u,2*(-1)**u)) for u in range(2) for c in range(3)}
    CF0={((0,c),(1,2)) for c in range(3)};CF1={e,compose(a,b)}
    need(A0==A1==AF and B0==BF0 and B1==BF1,'Actual embedded end fibers')
    need(C0==CF0 and C1==CF1,'Actual embedded medial fibers')
    need(A0&B0==C0 and A1&B1==C1 and C0&C1=={e},'Two unequal true intersections')
    need(not(A0&B0<=C0&C1),'Printed necessity actually fails')
    q=0
    for i in [1,2,1,2]:q=Q[q][i]
    x,y=pairs[q];witness=((0,2),(1,2))
    need(x==0 and H[y//2]==witness and witness in A0&B0 and witness not in C1,'Literal word witness')
    report={'status':'PASS','field':3,'affine_group_order':72,'input_flags':144,'mix_flags':2592,
            'input_path_intersections':pip,'mix_path_intersections':qip,'actual_input_automorphism_order':72,
            'actual_flag_orbits':2,'actual_medial_orbits':2,'medial_section_sizes':[4,6],
            'medial_orbit_sizes':[18,12],'uniform_facets':True,'uniform_vertex_figures':True,
            'class_I':[1,2],'regular_core_flags':len(monodromy),
            'even_facet_pairs':len(KA),'even_vertex_pairs':len(KB),
            'root_fiber_orders':[[len(A0),len(B0),len(C0)],[len(A1),len(B1),len(C1)]],
            'common_facet':A0==A1,'intersection_orders':[len(A0&B0),len(A1&B1)],
            'medial_group_intersection_order':len(C0&C1),'literal_610_condition':False,'literal_615_2_condition':False,
            'embedded_groups':{n:sorted(v) for n,v in [('A0',A0),('A1',A1),('B0',B0),('B1',B1),('C0',C0),('C1',C1)]},
            'witness':{'colors':[1,2,1,2],'affine_element':witness},
            'bounded_validation_only':True,'source_questions_answered':0}
    args.out.write_text(json.dumps(report,indent=2)+'\n');print(json.dumps({k:v for k,v in report.items() if
    k!='embedded_groups'},indent=2))
if __name__=='__main__':main()
```